1. Develop a Library Management Mini System.

1. Authentication Module

- Create Login & Registration pages
- Apply proper validation:
    1. Email format validation
    2. Password minimum 6 characters
- Store password in encrypted form
- Use Named Routes (URL-based navigation).

2. Book Module (CRUD)

- Add Book (Title, Author, ISBN, Quantity)
- Edit Book
- Delete Book
- Issue/Return Status
- Store data in browser local storage (JSON format).

Main.dart

```
import 'package:flutter/material.dart';
import 'routes/app_routes.dart';
import 'theme/app_theme.dart';

void main() {
  runApp(const LibraryApp());
}

class LibraryApp extends StatelessWidget {
```

```
  const LibraryApp({super.key});

  @override
  Widget build(BuildContext context) {

    return MaterialApp(

    debugShowCheckedModeBanner: false,

    title: "Library System",

      theme: AppTheme.lightTheme,

      initialRoute: "/",

      routes: AppRoutes.routes,

    );

  }
}
```

models/book_model.dart

```
class Book {

  String title;
  String author;
  String isbn;
  int quantity;
  bool issued;

  Book({
    required this.title,
    required this.author,
    required this.isbn,
    required this.quantity,
    this.issued=false
  });
```

```
  Map<String,dynamic> toJson(){
    return {
      "title":title,
      "author":author,
      "isbn":isbn,
      "quantity":quantity,
      "issued":issued
    };
  }

  factory Book.fromJson(Map json){

    return Book(
      title: json["title"],
      author: json["author"],
      isbn: json["isbn"],
      quantity: json["quantity"],
      issued: json["issued"]
    );

  }

}
```

providers/book_providers.dart

```
import 'package:flutter/material.dart';
import '../models/book_model.dart';
import '../services/storage_service.dart';

class BookProvider extends ChangeNotifier {

  List<Book> books = [];

  bool loading = true;

  BookProvider(){
    loadBooks();
  }
```

```
/// LOAD BOOKS FROM LOCAL STORAGE
void loadBooks() async {

  books = await StorageService.loadBooks();

  loading = false;

  notifyListeners();

}

/// ADD BOOK
void addBook(Book book){

  books.add(book);

  StorageService.saveBooks(books);

  notifyListeners();

}

/// DELETE BOOK
void deleteBook(int index){

  books.removeAt(index);

  StorageService.saveBooks(books);

  notifyListeners();

}

/// ISSUE / RETURN BOOK
void toggleIssue(int index){

  books[index].issued = !books[index].issued;

  StorageService.saveBooks(books);
```

```
    notifyListeners();

  }

  /// UPDATE / EDIT BOOK
  void updateBook(int index, Book updatedBook){

    books[index] = updatedBook;

    StorageService.saveBooks(books);

    notifyListeners();

  }

}
```

Routes/app_routes.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../providers/book_provider.dart';
import '../screens/login_screen.dart';
import '../screens/register_screen.dart';
import '../screens/dashboard_screen.dart';
import '../screens/add_book_screen.dart';
import '../screens/books_screen.dart';

class AppRoutes {
  static final BookProvider _bookProvider = BookProvider();

  static Map<String, WidgetBuilder> routes = {
    "/": (context) => const LoginScreen(),
    "/register": (context) => const RegisterScreen(),
    "/dashboard": (context) => const DashboardScreen(),

    "/add-book": (context) => ChangeNotifierProvider.value(
        value: _bookProvider,
```

```
        child: const AddBookScreen(),
      ),

    "/books": (context) => ChangeNotifierProvider.value(
        value: _bookProvider,
        child: const BooksScreen(),
      ),
  };
}
```

Screens/add_book_screen.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../models/book_model.dart';
import '../providers/book_provider.dart';

class AddBookScreen extends StatefulWidget {
  const AddBookScreen({super.key});

  @override
  State<AddBookScreen> createState() => _AddBookScreenState();
}

class _AddBookScreenState extends State<AddBookScreen> {
  final _formKey = GlobalKey<FormState>();

  final titleController = TextEditingController();
  final authorController = TextEditingController();
  final isbnController = TextEditingController();
  final quantityController = TextEditingController();

  @override
  void dispose() {
    titleController.dispose();
    authorController.dispose();
    isbnController.dispose();
    quantityController.dispose();
```

```
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text("Add Book"),
    ),
    body: SingleChildScrollView(
      padding: const EdgeInsets.all(24),
      child: Center(
        child: ConstrainedBox(
          constraints: const BoxConstraints(maxWidth: 550),
          child: Form(
            key: _formKey,
            child:  Column(
            children: [
              TextFormField(
                controller: titleController,
                decoration: const InputDecoration(
                labelText: "Title",
                  prefixIcon: Icon(Icons.book),
                ),
                validator: (value) {
                  if (value == null || value.trim().isEmpty) {
                    return "Title is required";
                  }
                  return null;
                },
              ),
              const SizedBox(height: 18),
              TextFormField(
                controller: authorController,
                decoration: const InputDecoration(
                labelText: "Author",
                  prefixIcon: Icon(Icons.person),
                ),
                validator: (value) {
```

```
    if (value == null || value.trim().isEmpty) {
      return "Author is required";
    }
    return null;
  },
),
const SizedBox(height: 18),
TextFormField(
  controller: isbnController,
  decoration: const InputDecoration(
  labelText: "ISBN",
    prefixIcon: Icon(Icons.qr_code_2),
  ),
  validator: (value) {
    if (value == null || value.trim().isEmpty) {
      return "ISBN is required";
    }
    return null;
  },
),
const SizedBox(height: 18),
TextFormField(
  controller: quantityController,
  keyboardType: TextInputType.number,
  decoration: const InputDecoration(
  labelText: "Quantity",
    prefixIcon: Icon(Icons.numbers),
  ),
  validator: (value) {
    if (value == null || value.trim().isEmpty) {
      return "Quantity is required";
    }
    final qty = int.tryParse(value);
    if (qty == null || qty <= 0) {
    return "Enter valid quantity";
    }
    return null;
  },
),
```

```
            const SizedBox(height: 30),
            SizedBox(
              width: double.infinity,
              height: 56,
              child: ElevatedButton(
                onPressed: () {
                  if (_formKey.currentState!.validate()) {
                    final newBook = Book(
                      title: titleController.text.trim(),
                      author: authorController.text.trim(),
                      isbn: isbnController.text.trim(),
                      quantity: int.parse(quantityController.text.trim()),
                    );

                    context.read<BookProvider>().addBook(newBook);

                    Navigator.pop(context);
                  }
                },
                child: const Text(
                  "Add Book",
                  style: TextStyle(
                  fontSize: 18,
                    fontWeight: FontWeight.w600,
                  ),
                ),
              ),
            ),
          ],
        ),
      ),
    ),
  ),
);
}
}
```

Screens/books_screen.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../providers/book_provider.dart';
import '../models/book_model.dart';

class BooksScreen extends StatelessWidget {
  const BooksScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<BookProvider>();

    if (provider.loading) {
      return const Scaffold(
      body: Center(
          child: CircularProgressIndicator(),
        ),
      );
    }

    if (provider.books.isEmpty) {
      return Scaffold(
        appBar: AppBar(title: const Text("Library Books")),
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Image.asset(
                "assets/images/empty_books.png",
                height: 200,
              ),
              const SizedBox(height: 20),
              const Text(
                "No Books Found",
                style: TextStyle(fontSize: 20),
              ),
            ],
          ),
        ),
```

```
  );
}

return Scaffold(
  appBar: AppBar(
    title: const Text("Library Books"),
  ),
  body: ListView.builder(
    padding: const EdgeInsets.all(16),
    itemCount: provider.books.length,
    itemBuilder: (context, index) {
      final book = provider.books[index];

      return Card(
        elevation: 4,
        margin: const EdgeInsets.only(bottom: 15),
        shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
        ),
        child: ListTile(
          contentPadding: const EdgeInsets.symmetric(
            horizontal: 16,
            vertical: 10,
          ),
          title: Text(
            book.title,
            style: const TextStyle(
              fontWeight: FontWeight.bold,
            ),
          ),
          subtitle: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const SizedBox(height: 6),
              Text("${book.author} • ISBN: ${book.isbn}"),
              const SizedBox(height: 6),
              Text("Quantity: ${book.quantity}"),
              const SizedBox(height: 8),
              Container(
```

```
        padding: const EdgeInsets.symmetric(
          horizontal: 10,
          vertical: 4,
        ),
        decoration: BoxDecoration(
          color: book.issued ? Colors.red : Colors.green,
          borderRadius: BorderRadius.circular(6),
        ),
        child: Text(
          book.issued ? "Issued" : "Available",
          style: const TextStyle(
            color: Colors.white,
            fontSize: 12,
          ),
        ),
      ),
    ],
  ),
  trailing: Row(
    mainAxisSize: MainAxisSize.min,
    children: [
      IconButton(
        tooltip: book.issued ? "Return Book" : "Issue Book",
        icon: const Icon(Icons.swap_horiz),
        onPressed: () {
          provider.toggleIssue(index);
        },
      ),
      IconButton(
        tooltip: "Edit Book",
        icon: const Icon(Icons.edit),
        onPressed: () {
        showEditDialog(context, index);
        },
      ),
      IconButton(
        tooltip: "Delete Book",
        icon: const Icon(Icons.delete, color: Colors.red),
        onPressed: () {
```

```
                provider.deleteBook(index);
              },
            ),
          ],
        ),
      ),
    );
  },
),
);
}
}

void showEditDialog(BuildContext context, int index) {
  final provider = context.read<BookProvider>();
  final book = provider.books[index];

  final titleController = TextEditingController(text: book.title);
  final authorController = TextEditingController(text: book.author);
  final isbnController = TextEditingController(text: book.isbn);
  final quantityController =
      TextEditingController(text: book.quantity.toString());

  showDialog(
    context: context,
    builder: (context) {
    return AlertDialog(
        title: const Text("Edit Book"),
        content: SingleChildScrollView(
        child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              TextField(
                controller: titleController,
                decoration: const InputDecoration(
                labelText: "Title",
                ),
              ),
              const SizedBox(height: 10),
```

```
      TextField(
        controller: authorController,
        decoration: const InputDecoration(
        labelText: "Author",
        ),
      ),
      const SizedBox(height: 10),
      TextField(
        controller: isbnController,
        decoration: const InputDecoration(
        labelText: "ISBN",
        ),
      ),
      const SizedBox(height: 10),
      TextField(
        controller: quantityController,
        keyboardType: TextInputType.number,
        decoration: const InputDecoration(
        labelText: "Quantity",
        ),
      ),
    ],
  ),
),
actions: [
  TextButton(
  onPressed: () {
      Navigator.pop(context);
    },
    child: const Text("Cancel"),
  ),
  ElevatedButton(
    onPressed: () {
      final quantity = int.tryParse(quantityController.text.trim());

      if (titleController.text.trim().isEmpty ||
          authorController.text.trim().isEmpty ||
          isbnController.text.trim().isEmpty ||
          quantity == null ||
```

```
            quantity <= 0) {
          return;
        }

        provider.updateBook(
          index,
          Book(
            title: titleController.text.trim(),
            author: authorController.text.trim(),
            isbn: isbnController.text.trim(),
            quantity: quantity,
            issued: book.issued,
          ),
        );

        Navigator.pop(context);
      },
      child: const Text("Update"),
    ),
  ],
);
},
);
}
```

Screens/dashboard_screen.dart

```
import 'package:flutter/material.dart';

class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  Widget card(BuildContext context, IconData icon, String title, String route) {
    return Expanded(
      child: InkWell(
        onTap: () {
          Navigator.pushNamed(context,  route);
        },
        borderRadius: BorderRadius.circular(12),
```

```
      child: Container(
        height: 120,
        decoration: BoxDecoration(
          color: Colors.white,
          borderRadius: BorderRadius.circular(12),
          boxShadow: const [
            BoxShadow(
              blurRadius: 10,
              color: Colors.black12,
            ),
          ],
        ),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Icon(icon, size: 35, color: Colors.deepPurple),
            const SizedBox(height: 8),
            Text(
              title,
              style: const TextStyle(
                fontWeight: FontWeight.w600,
                fontSize: 16,
              ),
            ),
          ],
        ),
      ),
    ),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      leading: const BackButton(),
      title: const Text("Dashboard"),
      actions: [
        Padding(
```

```
          padding: const EdgeInsets.only(right: 16),
          child: Image.asset(
          "assets/images/logo.jpg",
            height: 40,
          ),
        ),
      ],
    ),
    body: Padding(
      padding: const EdgeInsets.all(30),
      child: Row(
        children: [
          card(context, Icons.book, "View Books", "/books"),
          const SizedBox(width: 20),
          card(context, Icons.add, "Add Book", "/add-book"),
        ],
      ),
    ),
  );
 }
}
```

Screens/login_screen.dart

```
import 'package:flutter/material.dart';
import '../widgets/glass_card.dart';
import '../services/auth_service.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _formKey = GlobalKey<FormState>();

  final email = TextEditingController();
```

```
final password = TextEditingController();

bool isLoading = false;

final RegExp emailRegex = RegExp(
  r'^[\w\-\.]+@([\w\-]+\.)+[\w\-]{2,4}$',
);

@override
void dispose() {
  email.dispose();
  password.dispose();
  super.dispose();
}

Future<void> loginUser() async {
  if (!_formKey.currentState!.validate()) return;

  setState(() {
    isLoading = true;
  });

  try {
    final isRegistered = await AuthService.isUserRegistered();

    if (!isRegistered) {
      if (!mounted) return;

      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text("No account found. Please register first."),
        ),
      );

      return;
    }

    final isValid = await AuthService.loginUser(
      email: email.text.trim(),
```

```
      password: password.text.trim(),
    );

    if (!mounted) return;

    if (isValid) {
      await AuthService.setLoggedIn(true);
      Navigator.pushReplacementNamed(context, "/dashboard");
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
          content: Text("Invalid email or password"),
        ),
      );
    }
  } catch (e) {
    if (!mounted) return;

    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text("Login failed: $e"),
      ),
    );
  } finally {
    if (mounted) {
      setState(() {
      isLoading = false;
      });
    }
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Container(
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          colors: [Color(0xffc084fc), Color(0xff8b5cf6)],
```

```
  ),
),
child: Center(
  child: SizedBox(
  width: 350,
    child: GlassCard(
      child: Form(
      key: _formKey,
      child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Text(
              "Library Login",
              style: TextStyle(
              fontSize: 26,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 20),
            TextFormField(
              controller: email,
              decoration: const InputDecoration(
                labelText: "Email",
                prefixIcon: Icon(Icons.email),
              ),
              validator: (value) {
                if (value == null || value.trim().isEmpty) {
                  return "Email is required";
                }
                if (!emailRegex.hasMatch(value.trim())) {
                  return "Enter a valid email";
                }
                return null;
              },
            ),
            const SizedBox(height: 15),
            TextFormField(
              controller: password,
              obscureText: true,
```

```
  decoration: const InputDecoration(
    labelText: "Password",
    prefixIcon: Icon(Icons.lock),
  ),
  validator: (value) {
    if (value == null || value.isEmpty) {
      return "Password is required";
    }
    if (value.length < 6) {
      return "Password must be at least 6 characters";
    }
    return null;
  },
),
const SizedBox(height: 20),
SizedBox(
  width: double.infinity,
  child: ElevatedButton(
    onPressed: isLoading ? null : loginUser,
    child: isLoading
        ? const SizedBox(
            height: 20,
            width: 20,
            child: CircularProgressIndicator(
              strokeWidth: 2,
              color: Colors.white,
            ),
          )
        : const Text("Login"),
  ),
),
TextButton(
  onPressed: isLoading
      ? null
      : () {
          Navigator.pushNamed(context,  "/register");
        },
  child: const Text("Create Account"),
),
```

```
                ],
              ),
            ),
          ),
        ),
      ),
    ),
  );
 }
}
```

Screens/register_screen.dart

```
import 'package:flutter/material.dart';
import '../widgets/glass_card.dart';
import '../services/auth_service.dart';

class RegisterScreen extends StatefulWidget {
  const RegisterScreen({super.key});

  @override
  State<RegisterScreen> createState() => _RegisterScreenState();
}

class _RegisterScreenState extends State<RegisterScreen> {
  final _formKey = GlobalKey<FormState>();

  final nameController = TextEditingController();
  final emailController = TextEditingController();
  final passwordController = TextEditingController();
  final confirmPasswordController = TextEditingController();

  bool isLoading = false;

  final RegExp emailRegex = RegExp(
    r'^[\w\-\.]+@([\w\-]+\.)+[\w\-]{2,4}$',
  );

  @override
```

```
void dispose() {
  nameController.dispose();
  emailController.dispose();
  passwordController.dispose();
  confirmPasswordController.dispose();
  super.dispose();
}

Future<void> registerUser() async {
  if (!_formKey.currentState!.validate()) return;

  setState(() {
    isLoading = true;
  });

  try {
    await AuthService.registerUser(
      name: nameController.text.trim(),
      email: emailController.text.trim(),
      password: passwordController.text.trim(),
    );

    if (!mounted) return;

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text("Registration successful. Please login."),
      ),
    );

    Navigator.pop(context);
  } catch (e) {
    if (!mounted) return;

    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text("Registration failed: $e"),
      ),
    );
```

```
  } finally {
    if (mounted) {
      setState(() {
      isLoading = false;
      });
    }
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: Container(
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          colors: [Color(0xffc084fc), Color(0xff8b5cf6)],
        ),
      ),
      child: Center(
        child: SizedBox(
        width: 350,
          child: GlassCard(
            child: Form(
            key: _formKey,
            child: Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  const Text(
                    "Create Account",
                    style: TextStyle(
                    fontSize: 26,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                  const SizedBox(height: 20),
                  TextFormField(
                    controller: nameController,
                    decoration: const InputDecoration(
                    labelText: "Name",
```

```
    prefixIcon: Icon(Icons.person),
  ),
  validator: (value) {
    if (value == null || value.trim().isEmpty) {
      return "Name is required";
    }
    return null;
  },
),
const SizedBox(height: 15),
TextFormField(
  controller: emailController,
  decoration: const InputDecoration(
  labelText: "Email",
    prefixIcon: Icon(Icons.email),
  ),
  validator: (value) {
    if (value == null || value.trim().isEmpty) {
      return "Email is required";
    }
    if (!emailRegex.hasMatch(value.trim())) {
      return "Enter a valid email";
    }
    return null;
  },
),
const SizedBox(height: 15),
TextFormField(
  controller: passwordController,
  obscureText: true,
  decoration: const InputDecoration(
    labelText: "Password",
    prefixIcon: Icon(Icons.lock),
  ),
  validator: (value) {
    if (value == null || value.isEmpty) {
      return "Password is required";
    }
    if (value.length < 6) {
```

```
      return "Password must be at least 6 characters";
    }
    return null;
  },
),
const SizedBox(height: 15),
TextFormField(
  controller: confirmPasswordController,
  obscureText: true,
  decoration: const InputDecoration(
    labelText: "Confirm Password",
    prefixIcon: Icon(Icons.lock_outline),
  ),
  validator: (value) {
    if (value == null || value.isEmpty) {
      return "Confirm password is required";
    }
    if (value != passwordController.text) {
      return "Passwords do not match";
    }
    return null;
  },
),
const SizedBox(height: 20),
SizedBox(
  width: double.infinity,
  child: ElevatedButton(
    onPressed: isLoading ? null : registerUser,
    child: isLoading
        ? const SizedBox(
            height: 20,
            width: 20,
            child: CircularProgressIndicator(
              strokeWidth: 2,
              color: Colors.white,
            ),
          )
        : const Text("Register"),
  ),
```

```
                ),
                TextButton(
                  onPressed: isLoading
                      ? null
                      : () {
                          Navigator.pop(context);
                        },
                  child: const Text("Back to Login"),
                ),
              ],
            ),
          ),
        ),
      ),
    ),
  ),
  );
 }
}
```

Services/auth_service.dart

```
import 'dart:convert';
import 'package:crypto/crypto.dart';
import 'package:shared_preferences/shared_preferences.dart';

class AuthService {
  static const String _nameKey = "registered_name";
  static const String _emailKey = "registered_email";
  static const String _passwordKey = "registered_password";

  static String encryptPassword(String password) {
    final bytes = utf8.encode(password);
    final hash = sha256.convert(bytes);
    return hash.toString();
  }

  static Future<void> registerUser({
    required String name,
```

```
  required String email,
  required String password,
}) async {
  final prefs = await SharedPreferences.getInstance();

  await prefs.setString(_nameKey, name.trim());
  await prefs.setString(_emailKey, email.trim());
  await prefs.setString(_passwordKey, encryptPassword(password.trim()));
}

static Future<bool> isUserRegistered() async {
  final prefs = await SharedPreferences.getInstance();

  final savedEmail = prefs.getString(_emailKey);
  final savedPassword = prefs.getString(_passwordKey);

  return savedEmail != null &&
      savedEmail.isNotEmpty &&
      savedPassword != null &&
      savedPassword.isNotEmpty;
}

static Future<bool> loginUser({
  required String email,
  required String password,
}) async {
  final prefs = await SharedPreferences.getInstance();

  final savedEmail = prefs.getString(_emailKey);
  final savedPassword = prefs.getString(_passwordKey);

  if (savedEmail == null || savedPassword == null) {
    return false;
  }

  final enteredPasswordHash = encryptPassword(password.trim());

  return email.trim() == savedEmail && enteredPasswordHash == savedPassword;
}
```

```
  static Future<String?> getRegisteredName() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString(_nameKey);
  }

  static Future<String?> getRegisteredEmail() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString(_emailKey);
  }

  static Future<void> logout() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.remove("is_logged_in");
  }

  static Future<void> setLoggedIn(bool value) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool("is_logged_in", value);
  }

  static Future<bool> isLoggedIn() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getBool("is_logged_in") ?? false;
  }
}
```

Services/storange_service.dart

```
import 'dart:convert';
import 'package:shared_preferences/shared_preferences.dart';
import '../models/book_model.dart';

class StorageService {

  static const key = "books";

  static Future<List<Book>> loadBooks() async {
```

```
    final prefs = await SharedPreferences.getInstance();

    String? data = prefs.getString(key);

    if(data == null) return [];

    List jsonData = jsonDecode(data);

    return jsonData.map((e)=>Book.fromJson(e)).toList();

  }

  static Future saveBooks(List<Book> books) async {

    final prefs = await SharedPreferences.getInstance();

    List data = books.map((e)=>e.toJson()).toList();

    prefs.setString(key, jsonEncode(data));

  }

}
```

theme/app_theme.dart

```
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';

class AppTheme {

  static ThemeData lightTheme = ThemeData(

    primaryColor: Color(0xff8b5cf6),

    scaffoldBackgroundColor: Color(0xfff5f3ff),

    textTheme: GoogleFonts.poppinsTextTheme(),
```

```
    inputDecorationTheme: InputDecorationTheme(

      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(12),
      ),

      filled: true,
      fillColor: Colors.white,

    ),

  );
}
```

widgets/book_card.dart

```
import 'package:flutter/material.dart';
import '../models/book_model.dart';

class BookCard extends StatelessWidget {

  final Book book;
  final VoidCallback onDelete;
  final VoidCallback onToggle;
  final VoidCallback onEdit;

  const BookCard({
    super.key,
    required this.book,
    required this.onDelete,
    required this.onToggle,
    required this.onEdit,
  });

  @override
  Widget build(BuildContext context) {

    return Card(
```

```
elevation: 6,

shape: RoundedRectangleBorder(
  borderRadius: BorderRadius.circular(16),
),

child: ListTile(

  title: Text(book.title),

  subtitle: Text("${book.author} • ${book.isbn}"),

  trailing: Row(
    mainAxisSize: MainAxisSize.min,
    children: [

      Chip(
        label: Text(
          book.issued ? "Issued" : "Available",
        ),
      ),

      IconButton(
        icon: const Icon(Icons.edit),
        onPressed: onEdit,
      ),

      IconButton(
        icon: const Icon(Icons.swap_horiz),
        onPressed: onToggle,
      ),

      IconButton(
        icon: const Icon(Icons.delete),
        onPressed: onDelete,
      ),
    ],
  ),
),
```

```
    );
  }
}
```

widgets/glass_card.dart

```
import 'dart:ui';
import 'package:flutter/material.dart';

class GlassCard extends StatelessWidget {

final Widget child;

  const GlassCard({required this.child});

  @override
  Widget build(BuildContext context) {

    return ClipRRect(

      borderRadius: BorderRadius.circular(20),

      child: BackdropFilter(

        filter: ImageFilter.blur(sigmaX: 10, sigmaY: 10),

        child: Container(

          padding: EdgeInsets.all(25),

          decoration: BoxDecoration(

            color: Colors.white.withOpacity(0.2),

            borderRadius: BorderRadius.circular(20),

            border: Border.all(
              color: Colors.white.withOpacity(0.3),
            ),
```

```
        ),

        child: child,

      ),

    ),

  );
 }
}
```

widgets/gradient_button.dart

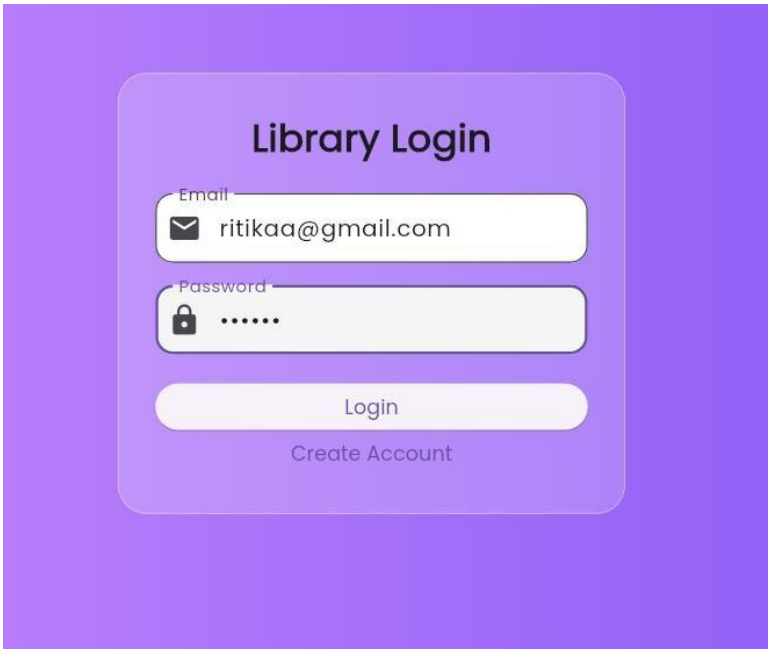
```
import 'package:flutter/material.dart';

class GradientButton extends StatelessWidget {

 final String text;
 final VoidCallback onPressed;

 const GradientButton({
  super.key,
  required this.text,
  required this.onPressed
 });

 @override
 Widget build(BuildContext context) {

  return InkWell(

   onTap: onPressed,

   child: Container(

    width: double.infinity,
    padding: const EdgeInsets.symmetric(
```

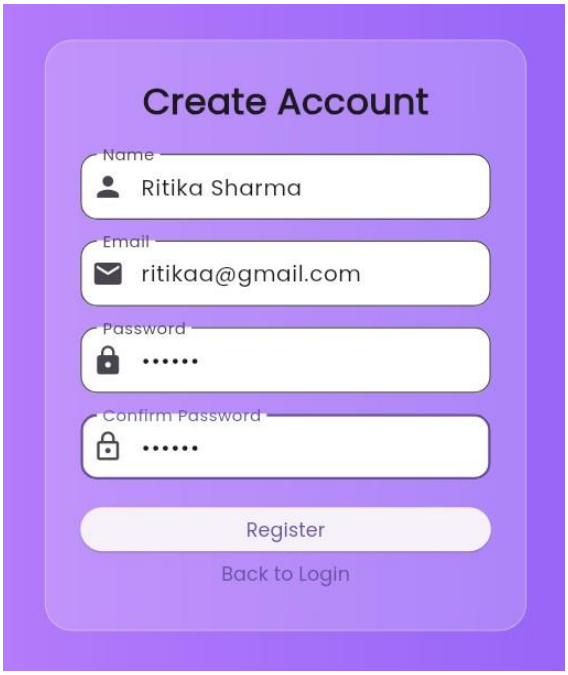
```
        vertical: 16,
      ),

      decoration: BoxDecoration(

        gradient: const LinearGradient(
          colors: [
            Color(0xffa78bfa),
            Color(0xff7c3aed)
          ],
        ),

        borderRadius: BorderRadius.circular(12),

        boxShadow: const [
          BoxShadow(
            color: Colors.black26,
            blurRadius: 10,
          )
        ],
      ),

      child: Center(
        child: Text(
        text,
          style: const TextStyle(
            color: Colors.white,
            fontWeight: FontWeight.bold,
            fontSize: 16
          ),
        ),
      ),
    ),
  );
 }
}
```

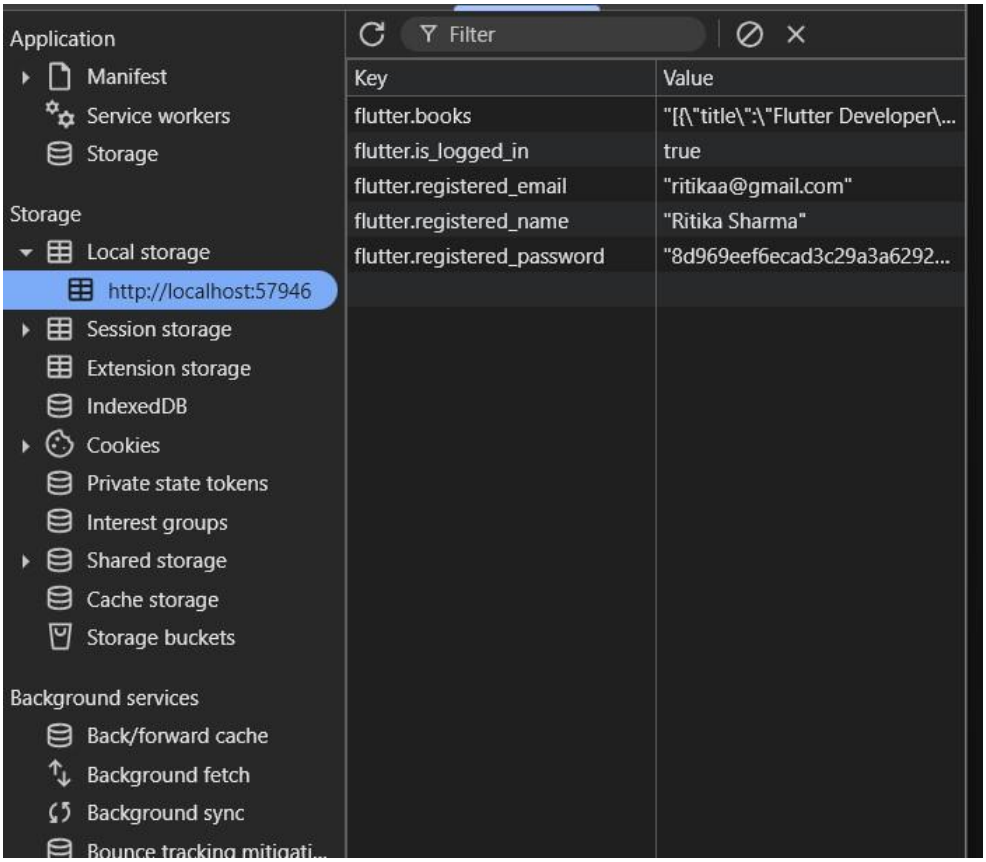
pubspec.yaml

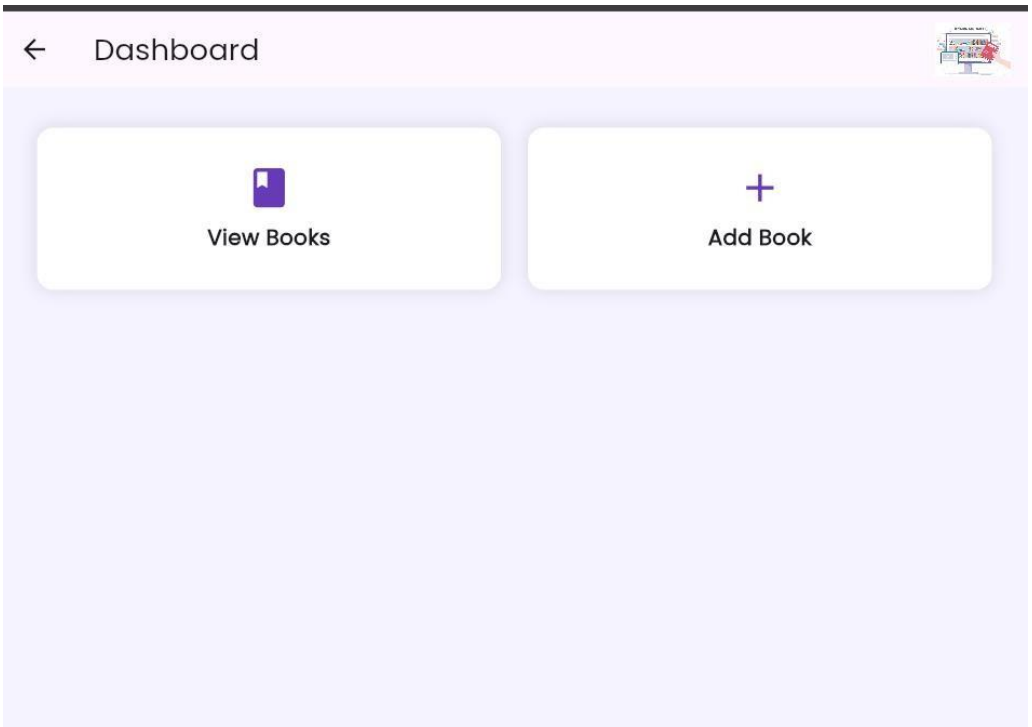
```
name: library_management_web
environment:
  sdk: ">=3.0.0 <4.0.0"
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.0.5
  google_fonts: ^6.1.0
  shared_preferences: ^2.2.2
  crypto: ^3.0.3
flutter:
  uses-material-design: true
 assets:
    - assets/images/
```

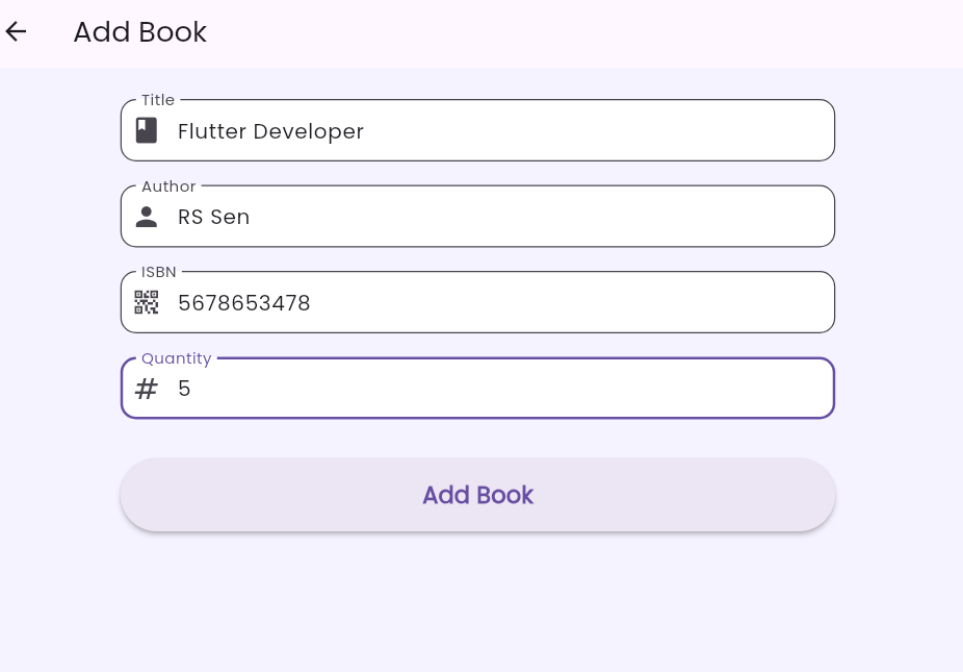
Output:

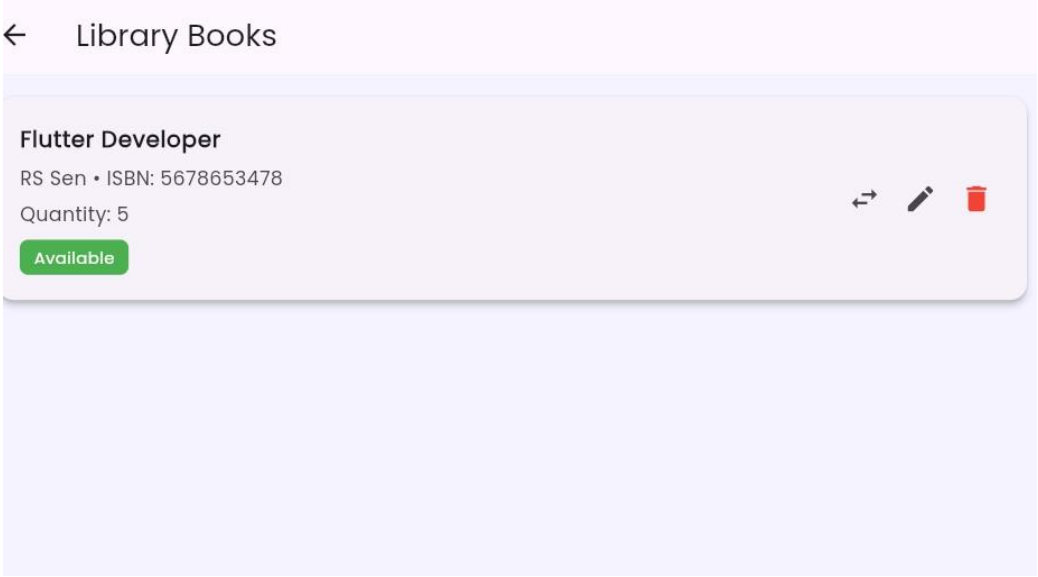

Library Login
Email
ritikaa@gmail.com
Password
Login
Create Account


Create Account
Name
Ritika Sharma
Email
ritikaa@gmail.com
Password
Confirm Password
Register
Back to Login

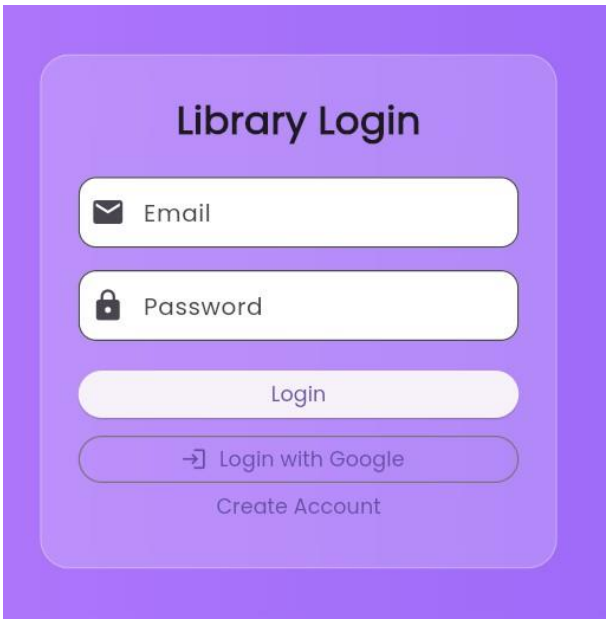

Application
Manifest
Service workers
Storage
Storage
Local storage
http://localhost:57946
Session storage
Extension storage
IndexedDB
Cookies
Private state tokens
Interest groups
Shared storage
Cache storage
Storage buckets
Background services
Back/forward cache
Background fetch
Background sync
Filter
Key
Value
flutter.books
"[{\"title\":\"Flutter Developer\...
flutter.is_logged_in
true
flutter.registered_email
"ritikaa@gmail.com"
flutter.registered_name
"Ritika Sharma"
flutter.registered_password
"8d969eef6ecad3c29a3a6292...

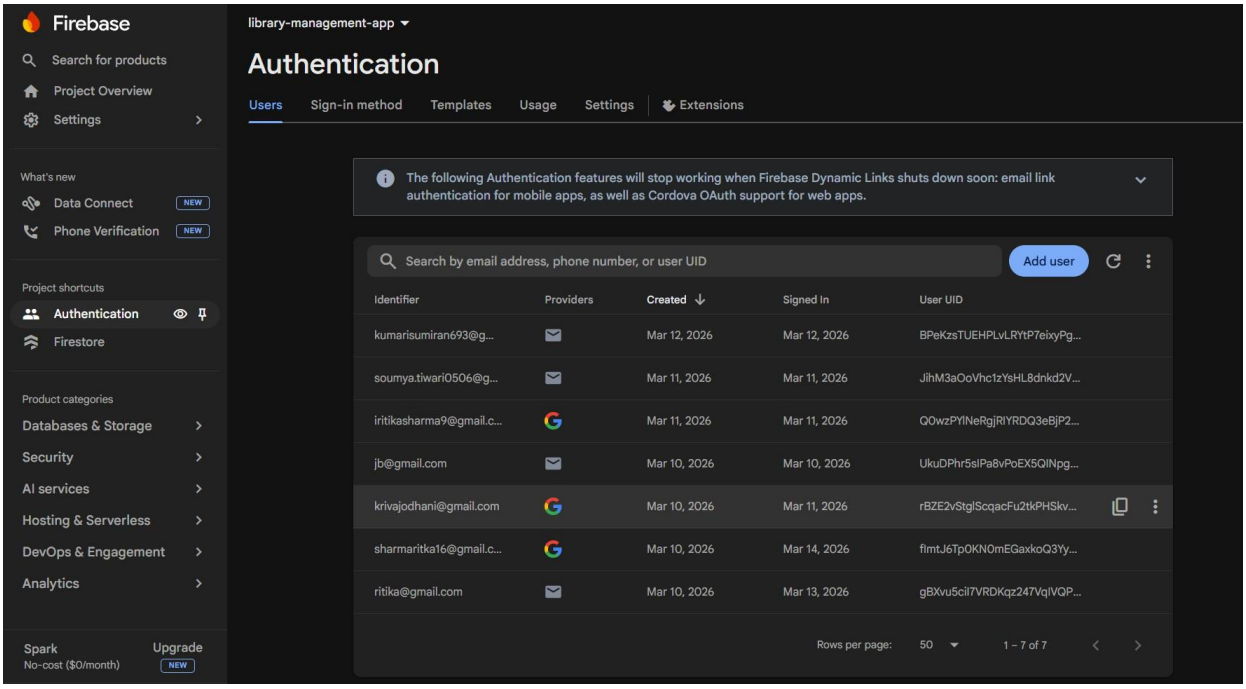

Dashboard
View Books
Add Book

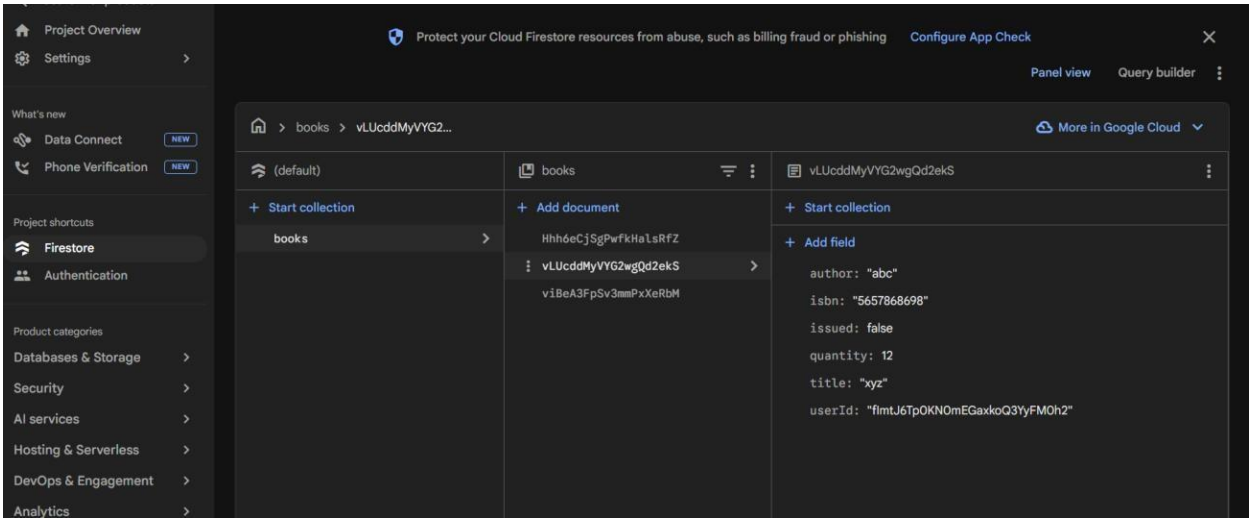

Add Book
Title
Flutter Developer
Author
RS Sen
ISBN
5678653478
Quantity
5
Add Book


Library Books
Flutter Developer
RS Sen • ISBN: 5678653478
Quantity: 5
Available

Firebase


Library Login
Email
Password
Login
Login with Google
Create Account

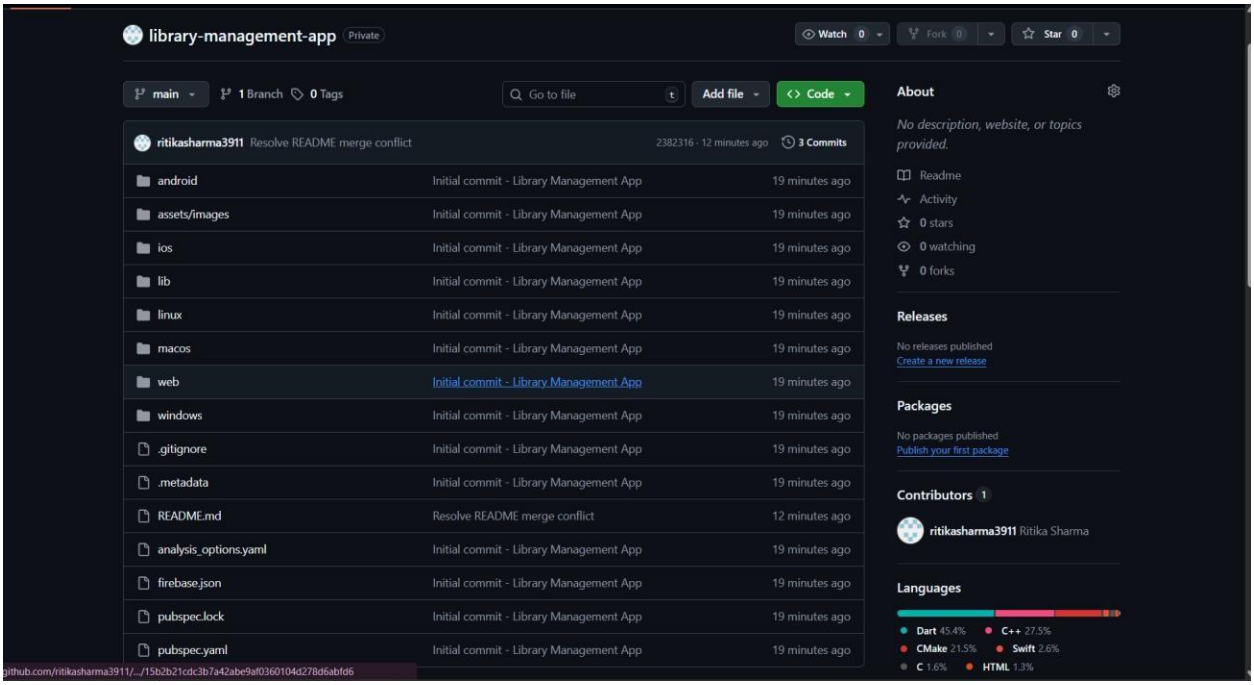

Firebase
Search for products
Project Overview
Settings
What's new
Data Connect NEW
Phone Verification NEW
Project shortcuts
Authentication
Firestore
Product categories
Databases & Storage
Security
AI services
Hosting & Serverless
DevOps & Engagement
Analytics
Spark
No-cost ($0/month)
Upgrade NEW
library-management-app
Authentication
Users
Sign-in method
Templates
Usage
Settings
Extensions
The following Authentication features will stop working when Firebase Dynamic Links shuts down soon: email link authentication for mobile apps, as well as Cordova OAuth support for web apps.
Search by email address, phone number, or user UID
Add user
Identifier
Providers
Created
Signed In
User UID
kumarisumiran693@g...
Mar 12, 2026
Mar 12, 2026
BPeKzsTUEHPLvLRYtP7eixyPg...
soumya.tiwari0506@g...
Mar 11, 2026
Mar 11, 2026
JihM3aOoVhc1zYsHL8dnkd2V...
iritikasharma9@gmail.c...
Mar 11, 2026
Mar 11, 2026
QOwzPYlNeRgjRlYRDQ3eBjP2...
jb@gmail.com
Mar 10, 2026
Mar 10, 2026
UkuDPhr5sIPa8vPoEX5QINpg...
krivajodhani@gmail.com
Mar 10, 2026
Mar 11, 2026
rBZE2vStglScqacFu2tkPHSkv...
sharmaritka16@gmail.c...
Mar 10, 2026
Mar 14, 2026
flmtJ6TpOKNOmEGaxkoQ3Yy...
ritika@gmail.com
Mar 10, 2026
Mar 13, 2026
gBXvu5cil7VRDKqz247VqlVQP...
Rows per page: 50
1 – 7 of 7


Project Overview
Settings
What's new
Data Connect NEW
Phone Verification NEW
Project shortcuts
Firestore
Authentication
Product categories
Databases & Storage
Security
AI services
Hosting & Serverless
DevOps & Engagement
Analytics
Protect your Cloud Firestore resources from abuse, such as billing fraud or phishing
Configure App Check
Panel view
Query builder
books > vLUcddMyVYG2...
More in Google Cloud
(default)
+ Start collection
books
books
+ Add document
Hhh6eCjSgPwfkHalsRfZ
vLUcddMyVYG2wgQd2ekS
viBeA3FpSv3mmPxXeRbM
vLUcddMyVYG2wgQd2ekS
+ Start collection
+ Add field
author: "abc"
isbn: "5657868698"
issued: false
quantity: 12
title: "xyz"
userId: "flmtJ6Tp0KNOmEGaxkoQ3YyFM0h2"

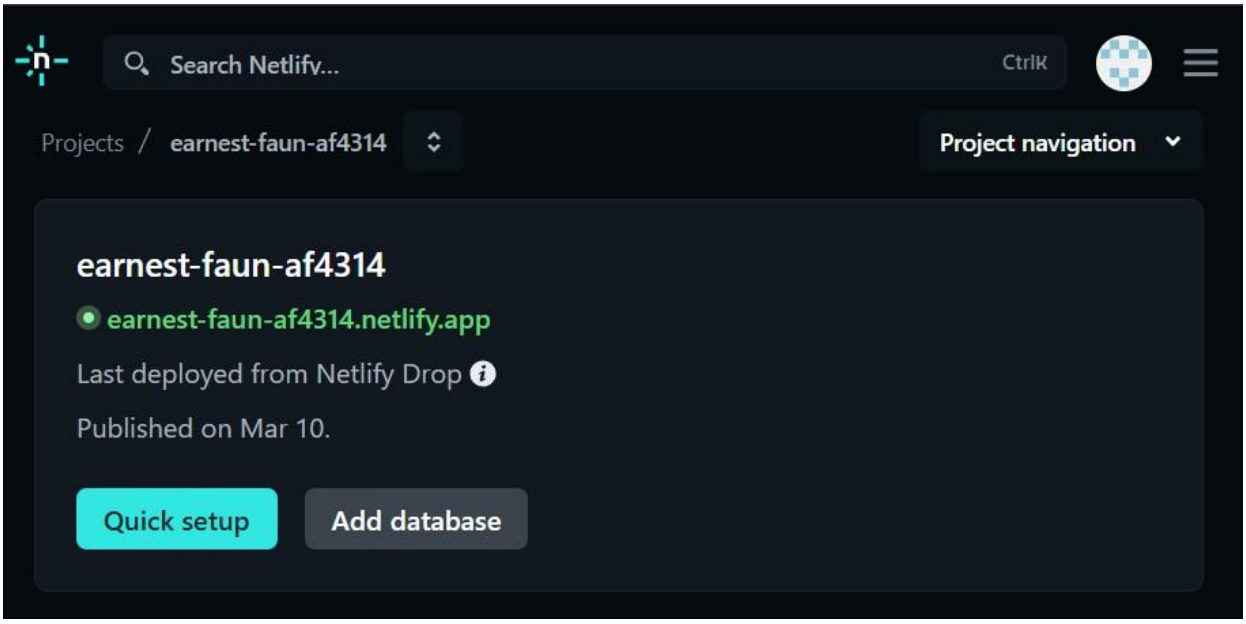
GitHub

Github link - https://github.com/ajaysonavane/library-management-app.git

Github Scanner -


library-management-app
Private
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
Add file
Code
ritikasharma3911 Resolve README merge conflict
2382316 · 12 minutes ago
3 Commits
android Initial commit - Library Management App 19 minutes ago
assets/images Initial commit - Library Management App 19 minutes ago
ios Initial commit - Library Management App 19 minutes ago
lib Initial commit - Library Management App 19 minutes ago
linux Initial commit - Library Management App 19 minutes ago
macos Initial commit - Library Management App 19 minutes ago
web Initial commit - Library Management App 19 minutes ago
windows Initial commit - Library Management App 19 minutes ago
.gitignore Initial commit - Library Management App 19 minutes ago
.metadata Initial commit - Library Management App 19 minutes ago
README.md Resolve README merge conflict 12 minutes ago
analysis_options.yaml Initial commit - Library Management App 19 minutes ago
firebase.json Initial commit - Library Management App 19 minutes ago
pubspec.lock Initial commit - Library Management App 19 minutes ago
pubspec.yaml Initial commit - Library Management App 19 minutes ago
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 1
ritikasharma3911 Ritika Sharma
Languages
Dart 45.4%
C++ 27.5%
CMake 21.5%
Swift 2.6%
C 1.6%
HTML 1.3%

Netlify


Search Netlify...
CtrlK
Projects / earnest-faun-af4314
Project navigation
earnest-faun-af4314
earnest-faun-af4314.netlify.app
Last deployed from Netlify Drop
Published on Mar 10.
Quick setup
Add database

Netlify Scanner